

# NetTopo contra sistemas reais

Metodologia, resultados por cenário, análise por fabricante e protocolo, defeitos com localização no código, desempenho, qualidade, segurança e plano de melhorias da primeira simulação do NetTopo em laboratório Containerlab.

|                             |                                            |                             |                    |                                               |                                                  |
|-----------------------------|--------------------------------------------|-----------------------------|--------------------|-----------------------------------------------|--------------------------------------------------|
| DOCUMENTO<br>NT-AVL-2026-02 | VERSÃO AVALIADA<br>NetTopo 0.1.0 · f2823ea | EXECUÇÃO<br>run 36256925269 | DATA<br>26/09/2026 | RUNNER<br>ubuntu-24.04 · 4 vCPU · 16 GB · KVM | PÚBLICO<br>Engenharia de redes e desenvolvimento |
|-----------------------------|--------------------------------------------|-----------------------------|--------------------|-----------------------------------------------|--------------------------------------------------|

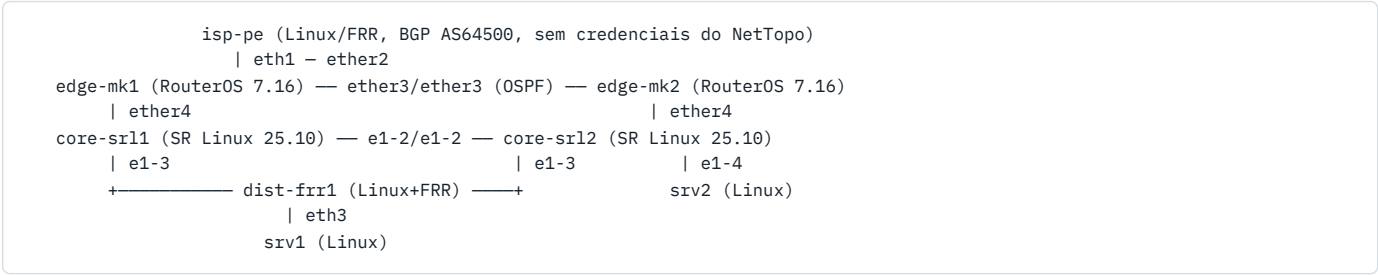
## Objetivo e escopo

Validar a ferramenta contra sistemas operacionais de rede reais, e não mais contra o coletor simulado usado nos testes unitários. A avaliação cobre descoberta, identificação de fabricante, leitura dos dados (parsing), montagem do grafo, saídas, API, interface web e linha de comando.

A versão 0.1.0 foi avaliada **como está**. Houve uma única correção durante a simulação, porque impedia a execução (defeito D0, abaixo). Todas as demais falhas estão registradas como recomendação, sem correção.

## Ambiente

| ITEM         | VERSÃO / CONFIGURAÇÃO                                                      |
|--------------|----------------------------------------------------------------------------|
| Execução     | GitHub Actions, ubuntu-24.04, 4 vCPU, 15 GiB, /dev/kvm habilitado          |
| Orquestração | Containerlab 0.79.0                                                        |
| MikroTik     | RouterOS 7.16.2 CHR em VM QEMU/KVM, imagem gerada com vrnetlab v0.21.0     |
| Nokia        | SR Linux 25.10.5 (ghcr.io/nokia/srlinux), NETCONF + OpenConfig habilitados |
| Linux        | Debian 12, FRR, lldpd com AgentX (LLDP-MIB via net-snmp), OpenSSH          |
| NetTopo      | 0.1.0, Python 3.12, commit f2823ea                                         |



Gerência em 172.20.20.0/24, alcançável pelo servidor de descoberta. Os endereços dos links de dados (10.x, 192.168.x, 200.200.10.0/30) não são alcançáveis por ele, como costuma acontecer com interfaces de trânsito numa rede real.

Uma rodada completa levou 16 min 19 s: build do CHR 2 min 55 s, deploy 1 min 37 s, boot e configuração 52 s, cenários 9 min 08 s e qualidade de código 48 s.

## Metodologia

O arquivo lab/ground\_truth.yaml descreve o que deveria ser encontrado: 7 equipamentos coletáveis, 1 roteador de operadora que deve aparecer como "não coletado", 9 links físicos com as interfaces nativas de cada lado e 2 adjacências de roteamento. O script lab/scripts/score.py compara cada descoberta com esse gabarito:

- Equipamentos coletados:** equipamentos coletáveis presentes e coletados (não apenas vistos por vizinhos).
- Links encontrados:** pares de equipamentos do gabarito ligados por ao menos um link físico.

- **Precisão de links:** links corretos ÷ links desenhados. Links a mais entre o mesmo par (duplicados ou da rede de gerência) e links para pares inexistentes contam como erro.
- **Portas corretas:** links encontrados com o nome de interface certo nas duas pontas.
- **Fabricante e papel:** acerto sobre os equipamentos coletados.
- **Coletas repetidas:** eventos de coleta concluída atribuídos ao mesmo equipamento final.

Antes dos cenários, `lab/scripts/dump_raw.py` executa em cada equipamento os comandos do perfil de fabricante do NetTopo e mede quantos registros cada parser extrai das saídas reais.

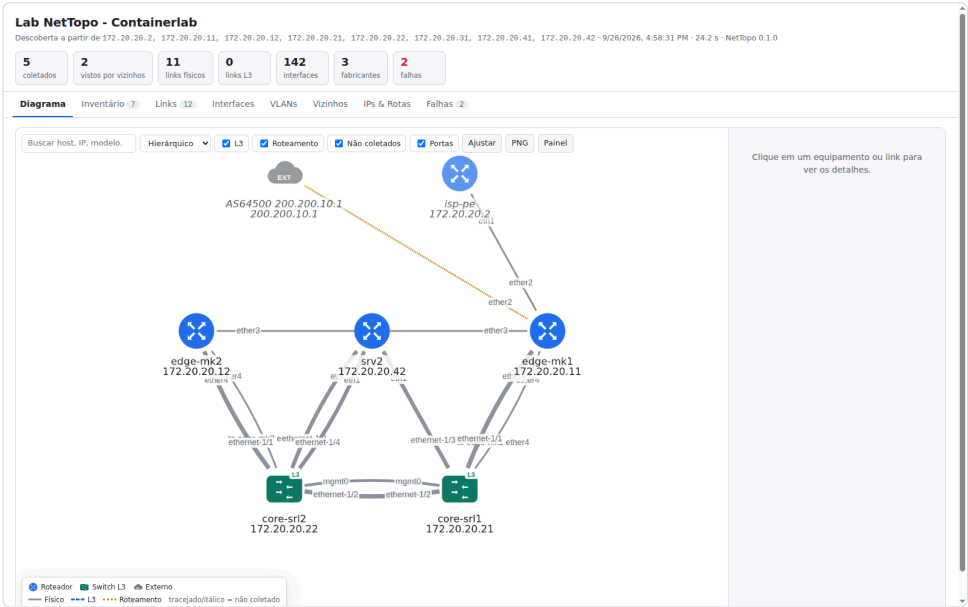
## Resultados por cenário

| CENÁRIO                            | EQUIPAMENTOS COLETADOS     | LINKS ENCONTRADOS          | PORTAS CORRETAS             | TEMPO (S) |
|------------------------------------|----------------------------|----------------------------|-----------------------------|-----------|
| 01 Semente única, todos os métodos | <div><div></div></div> 4/7 | <div><div></div></div> 7/9 | <div><div></div></div> 86%  | 24.5      |
| 02 Semente única + DNS             | <div><div></div></div> 4/7 | <div><div></div></div> 6/9 | <div><div></div></div> 67%  | 23.9      |
| 03 Inventário de IPs de gerência   | <div><div></div></div> 5/7 | <div><div></div></div> 6/9 | <div><div></div></div> 50%  | 24.2      |
| 04 Somente SNMP v2c                | <div><div></div></div> 1/7 | <div><div></div></div> 3/9 | <div><div></div></div> 67%  | 0.8       |
| 05 Somente SNMPv3                  | <div><div></div></div> 3/7 | <div><div></div></div> 4/9 | <div><div></div></div> 100% | 10.9      |
| 06 Somente SSH                     | <div><div></div></div> 0/7 | <div><div></div></div> 0/9 | —                           | 118.1     |
| 07 Somente APIs (REST/NETCONF)     | <div><div></div></div> 6/7 | <div><div></div></div> 9/9 | <div><div></div></div> 89%  | 13.6      |
| 08 Credenciais erradas primeiro    | <div><div></div></div> 4/7 | <div><div></div></div> 7/9 | <div><div></div></div> 86%  | 44.0      |
| 09 Profundidade 1 + exclusão       | <div><div></div></div> 2/7 | <div><div></div></div> 5/9 | <div><div></div></div> 80%  | 23.7      |
| 10 Inventário sequencial           | <div><div></div></div> 5/7 | <div><div></div></div> 7/9 | <div><div></div></div> 71%  | 41.7      |
| 11 Após falha injetada             | <div><div></div></div> 5/7 | <div><div></div></div> 6/9 | <div><div></div></div> 67%  | 24.3      |

Barras em escala de 0 a 100% do gabarito (7 equipamentos coletáveis, 9 links físicos). Fonte: `metrics.json` do run 36256925269-1.

Leitura dos cenários:

- **01 e 02.** `edge-mk2` só é anunciado com o IP da interface de trânsito `10.255.0.2` (MNDP/LLDP do RouterOS). A ferramenta tenta esse IP, falha e não tenta o nome. Com DNS configurado o resultado é o mesmo, porque o nome só é resolvido quando o vizinho não anuncia IP nenhum (D3).
- **03.** Com todos os IPs de gerência como sementes, todos os equipamentos respondem, mas `dist-frr1`, `srv1` e `srv2` viram um único registro (D1). Os links desses hosts passam a apontar para o registro fundido, o que derruba a precisão para 55% e o acerto de portas para 50%.
- **04.** O RouterOS não preenche o endereço de gerência na LLDP-MIB. Sem IP, a descoberta para no primeiro salto (D9), mas é muito rápida: 0,8 s.
- **05.** SNMPv3 authPriv (SHA/AES) funcionou nos três hosts Linux, os únicos com usuário v3 no laboratório. Como no 07, os hosts Linux não se fundem: a série falsa só aparece na coleta por SSH (D1).
- **06.** O autodetect do Netmiko não reconhece o RouterOS. Sem perfil, o coletor SSH desiste da semente e nada é descoberto após 118 s de tentativas (D4).
- **07.** Melhor resultado: REST no RouterOS, NETCONF/OpenConfig no SR Linux e SNMP nos hosts Linux, com SNMP apenas para identificar o fabricante. Sem a coleta SSH dos hosts Linux, não há fusão.
- **08.** Três credenciais erradas antes das corretas custaram 19,5 s e não mudaram o resultado. Depois do primeiro acerto, a credencial que funcionou passa para o início da lista.
- **09.** Profundidade 1 e exclusão da rede dos servidores funcionaram como configurado.
- **10.** Sequencial (1 coleta por vez) levou 41,7 s contra 24,2 s com 16 em paralelo (1,7×).
- **11.** `srv2` desligado aparece em falhas e o link `core-sr12:e1-3` + `dist-frr1:eth2` derrubado some como link físico, mas volta como link L3 inferido, porque a sub-rede /30 continua configurada.



Relatório HTML gerado no cenário 03. O registro "srv2" representa três hosts Linux fundidos; há duas linhas entre edge-mk1 e core-srl1 e entre edge-mk2 e core-srl2, e um link mgmt0 entre os switches.

## Matriz fabricante × método

| SISTEMA              | SNMP V2C           | SNMPV3                   | SSH/CLI          | APIS                     |
|----------------------|--------------------|--------------------------|------------------|--------------------------|
| MikroTik CHR 7.16    | <div>parcial</div> | <div>não se aplica</div> | <div>falha</div> | <div>parcial</div>       |
| Nokia SR Linux 25.10 | <div>falha</div>   | <div>não se aplica</div> | <div>falha</div> | <div>funciona</div>      |
| Linux / FRR          | <div>falha</div>   | <div>funciona</div>      | <div>falha</div> | <div>não se aplica</div> |

Coleta por método isolado. “Não se aplica”: o método não existe/não foi configurado nesse sistema no laboratório.

A matriz considera só os cenários de método isolado (04 a 07), a partir da mesma semente `edge-mk1`, exceto o 05, que parte do inventário. "Falha" em SNMP v2c para SR Linux e Linux não significa que o SNMP desses sistemas não funcione. Eles nem chegaram a ser consultados, porque a descoberta parou no primeiro salto (D9). Nos cenários completos, o SNMP complementou os dados dos três sistemas.

## Parsing com saídas reais

| EQUIPAMENTO | IDENTIFICAÇÃO                                         | HOSTNAME                                                 | INTERFACES                                                 | IPS                                                        | LLDP                                     |
|-------------|-------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------|------------------------------------------|
| edge-mk1    | <div>funciona</div> <div>1 reg · mikrotik_facts</div> | <div>funciona</div> <div>1 reg · mikrotik_identity</div> | <div>funciona</div> <div>5 reg · mikrotik_interfaces</div> | <div>funciona</div> <div>4 reg · mikrotik_ip_address</div> | <div>não se aplica</div>                 |
| core-srl1   | <div>funciona</div> <div>1 reg · kv_colon</div>       | <div>não se aplica</div>                                 | <div>não se aplica</div>                                   | <div>falha</div> <div>0 reg · none</div>                   | <div>falha</div> <div>0 reg · none</div> |
| dist-frr1   | <div>funciona</div> <div>1 reg · linux_facts</div>    | <div>funciona</div> <div>1 reg · first_line</div>        | <div>funciona</div> <div>4 reg · linux_links</div>         | <div>funciona</div> <div>4 reg · linux_addrs</div>         | <div>falha</div> <div>0 reg · none</div> |

Saídas reais dos comandos do perfil de cada fabricante, interpretadas pelos parsers do NetTopo (registros extraídos · parser usado). Fonte: `raw/parse-matrix.json`.

- RouterOS 7:** os parsers próprios leem quase tudo. Rotas: 1 de 6, porque o `print terse` do RouterOS 7 não numera rotas dinâmicas e o agrupador de registros só inicia um registro novo numa linha com índice (D7).
- SR Linux:** as saídas de `show system lldp neighbor` e `show interface brief` são tabelas com bordas ASCII; nenhum template TextFSM existe para a plataforma e o parser genérico espera "chave: valor" (D8). Nos cenários completos isso não aparece porque o SR Linux foi coletado por NETCONF.
- Linux:** `lldpctl` não está no `PATH` de usuários sem privilégio no Debian (fica em `/usr/sbin`), e a saída é `command not found` (D11). O LLDP dos hosts Linux veio pela LLDP-MIB via SNMP.

# Defeitos encontrados

Severidade **alta**: distorce o inventário ou o desenho, ou impede um modo de uso. **Média**: dado incompleto ou errado com contorno. **Baixa**: cosmético ou de mensagem.

| ID  | SEV.  | DEFEITO                                                                                                                                                                                                       | ONDE                                                                                                                                                                                               | EVIDÊNCIA                                                                                                                        |
|-----|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| D0  | ALTA  | <code>.gitignore</code> com <code>output/</code> excluía <code>src/nettopo/output</code> ; quem clonasse o repositório teria a ferramenta sem o pacote de saídas. <b>Corrigido</b> ( <code>/output/</code> ). | <code>.gitignore</code>                                                                                                                                                                            | Run 36255328422: <code>ModuleNotFoundError: nettopo.output</code>                                                                |
| D1  | ALTA  | Hosts Linux fundidos: o parser usa o nome do produto DMI ("Virtual Machine") como número de série quando a série não pode ser lida, e a deduplicação confia na série.                                         | <code>src/nettopo/vendors/parsers/linux.py:103</code> , <code>src/nettopo/discovery/identity.py:73</code>                                                                                          | 3 hosts em 1 registro em 01, 02, 03, 08 e 10; 2 em 1 no 11                                                                       |
| D2  | ALTA  | Links duplicados: o RouterOS expõe em <code>interface-name</code> a descrição da porta do vizinho (SR Linux: "to edge-mk1 ether4"), e a fusão de links não reconhece que é a mesma porta.                     | <code>src/nettopo/vendors/parsers/mikrotik.py:65</code> , <code>src/nettopo/collectors/rest.py</code> (driver MikroTik), <code>src/nettopo/topology/builder.py</code> ( <code>merge_links</code> ) | Link a mais RouterOS ↔ SR Linux em 8 dos 11 cenários (todos em que os dois foram coletados)                                      |
| D3  | ALTA  | Sem alternativa quando o IP anunciado não responde; o nome só é resolvido quando nenhum IP é anunciado.                                                                                                       | <code>src/nettopo/discovery/scope.py:65-67</code>                                                                                                                                                  | <code>edge-mk2</code> não coletado em 01 e 02 (DNS sem efeito)                                                                   |
| D4  | ALTA  | Identificação de fabricante depende de SNMP ou do autodetect do Netmiko, que não reconhece o RouterOS; sem perfil, o coletor SSH desiste.                                                                     | <code>src/nettopo/collectors/orchestrator.py:60-62</code> , <code>src/nettopo/collectors/ssh.py:106</code>                                                                                         | Cenário 06: 0/7 em 118 s                                                                                                         |
| D5  | MÉDIA | LLDP recebido em interfaces de gerência (mgmt0) vira link físico.                                                                                                                                             | <code>src/nettopo/topology/builder.py:170</code>                                                                                                                                                   | Link <code>core-srl1:mgmt0</code> ↔ <code>core-srl2:mgmt0</code> em 8 dos 11 cenários (todos em que os SR Linux foram coletados) |
| D6  | MÉDIA | Inferência L3 aceita dois equipamentos com o <b>mesmo</b> IP numa /30 (gerência interna NAT do CHR, <code>172.31.255.30/30</code> ) e grava a sub-rede errada no link.                                        | <code>src/nettopo/topology/enrich.py:29-50</code>                                                                                                                                                  | Sub-rede <code>172.31.255.28/30</code> no link <code>edge-mk1</code> ↔ <code>edge-mk2</code> nos cenários 03, 10 e 11            |
| D7  | MÉDIA | Rotas do RouterOS 7 sem índice são descartadas.                                                                                                                                                               | <code>src/nettopo/vendors/parsers/mikrotik.py:12,24</code>                                                                                                                                         | 1 de 6 rotas                                                                                                                     |
| D8  | MÉDIA | SR Linux por CLI sem parser para tabelas.                                                                                                                                                                     | <code>src/nettopo/vendors/profiles/datacenter_campus.yaml:144</code>                                                                                                                               | LLDP e interfaces: 0 registros                                                                                                   |
| D9  | MÉDIA | Vizinho sem IP de gerência (LLDP-MIB do RouterOS) não é seguido; o MAC do chassis poderia ser resolvido pela tabela ARP já coletada.                                                                          | <code>src/nettopo/discovery/scope.py</code>                                                                                                                                                        | Cenário 04: 1/7                                                                                                                  |
| D10 | MÉDIA | Adjacência OSPF nunca detectada: o driver REST do RouterOS não consulta vizinhos OSPF/BGP e o RouterOS não os expõe por SNMP no laboratório.                                                                  | <code>src/nettopo/collectors/rest.py</code> ( <code>MikrotikRestDriver.paths</code> )                                                                                                              | 0 de 11 cenários                                                                                                                 |
| D11 | BAIXA | Comando <code>lldpctl</code> fora do <code>PATH</code> de usuários comuns.                                                                                                                                    | <code>src/nettopo/vendors/profiles/brasil_linux.yaml:70</code>                                                                                                                                     | <code>command not found</code>                                                                                                   |
| D12 | BAIXA | O vizinho BGP da operadora vira um nó separado ("AS64500 200.200.10.1") em vez de se juntar ao nó já visto por LLDP.                                                                                          | <code>src/nettopo/topology/enrich.py</code> ( <code>routing_adjacencies</code> )                                                                                                                   | Nó inesperado em 8 dos 11 cenários                                                                                               |
| D13 | BAIXA | Inventário incompleto: série e modelo ausentes para SR Linux e série ausente para o CHR.                                                                                                                      | coletores NETCONF/REST                                                                                                                                                                             | Campos vazios no inventário                                                                                                      |
| D14 | BAIXA | Mensagem enganosa: falha de autenticação SSH é reportada como "perfil sem driver Netmiko".                                                                                                                    | <code>src/nettopo/collectors/orchestrator.py</code>                                                                                                                                                | <code>isp-pe</code> em 7 dos 11 cenários                                                                                         |

Consequência de D1 que merece destaque: o equipamento que "sobrevive" à fusão depende da ordem de chegada das coletas (`dist-frr1` em 01, `srv2` em 02 e 03). O mesmo laboratório produz inventários diferentes a cada execução, e a comparação entre descobertas (`nettopo diff`) acusa equipamentos "adicionados" e "removidos" que não mudaram.

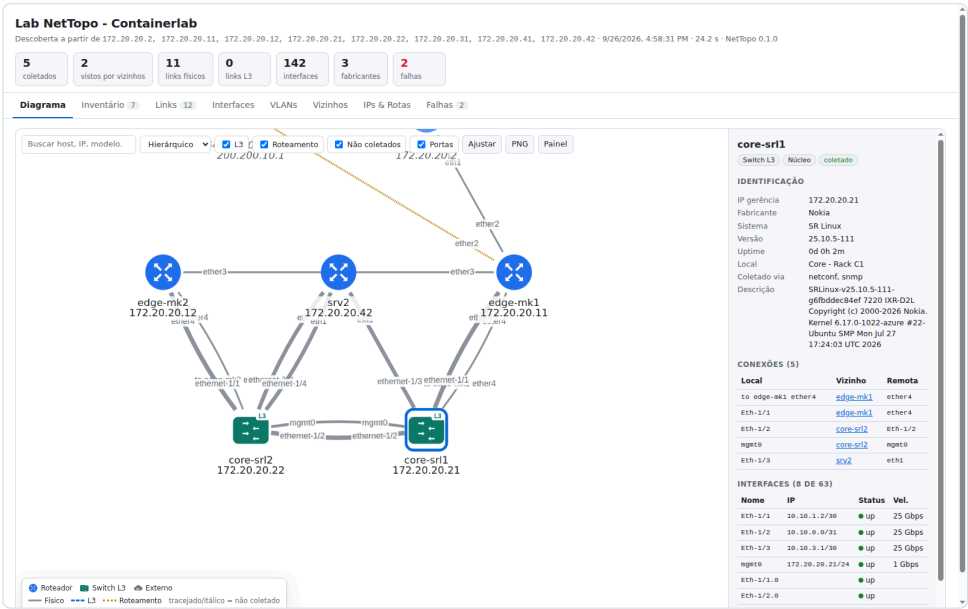
## Desempenho

| MEDIDA                                                      | RESULTADO                                                         |
|-------------------------------------------------------------|-------------------------------------------------------------------|
| Descoberta completa, 8 equipamentos, 16 coletas em paralelo | 24,2 a 24,5 s                                                     |
| Mesma descoberta, 1 coleta por vez                          | 41,7 s (1,7×)                                                     |
| Somente SNMP v2c (1 equipamento alcançado)                  | 0,8 s                                                             |
| Somente APIs (6 equipamentos)                               | 13,6 s                                                            |
| Custo de 3 credenciais erradas antes das certas             | +19,5 s                                                           |
| Custo de um alvo inacessível                                | porta a porta (22, 830, 443, 80) + tentativas SNMP por credencial |
| Somente SSH (autodetect falhando)                           | 118,1 s                                                           |

O tempo é dominado por alvos inacessíveis e autenticações que falham. Numa rede com centenas de equipamentos, D3 (IPs não roteáveis) e D4 (autodetect) multiplicam esse custo. Sugestões: timeouts separados para varredura de portas e SNMP, cache negativo por IP e limite de credenciais por tentativa.

## API, interface web e linha de comando

| VERIFICAÇÃO                                                                                | RESULTADO                                                                                            |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>POST /api/discoveries</code> + WebSocket até o fim                                   | concluída em 24,6 s; eventos recebidos: 1 início, 9 coletas iniciadas, 7 concluídas, 2 falhas, 1 fim |
| Exportações pela API                                                                       | HTML 121 KB, draw.io 39 KB, Excel 30 KB, Markdown (zip) 7 KB, JSON 100 KB; todas HTTP 200            |
| Senhas no banco SQLite                                                                     | nenhuma encontrada (busca pelas três senhas do laboratório)                                          |
| CLI <code>vendors</code> , <code>discover</code> , <code>report</code> , <code>diff</code> | todos com código de saída 0                                                                          |
| Capturas da interface                                                                      | 5 telas geradas pelo Chrome headless                                                                 |



Painel de detalhes do core-sr11 no relatório HTML: identificação, conexões e interfaces coletadas por NETCONF. Modelo e série vazios (D13).

## Qualidade de código

| MÉTRICA                        | RESULTADO    |
|--------------------------------|--------------|
| Testes ( <code>pytest</code> ) | 46 aprovados |

| MÉTRICA                       | RESULTADO                                                                                                      |
|-------------------------------|----------------------------------------------------------------------------------------------------------------|
| Cobertura                     | 71% de 3.875 instruções                                                                                        |
| Lint ( <code>ruff</code> )    | sem ocorrências                                                                                                |
| Tipagem ( <code>mypy</code> ) | 22 erros em 10 arquivos, principalmente valores opcionais ( <code>None</code> ) usados onde o tipo exige valor |
| <code>bandit</code>           | 1 alto, 9 médios, 5 baixos                                                                                     |

Triagem do `bandit` :

- **Alto, B324 (MD5):** `hashlib.md5` em `collectors/simulated.py:27` gera MACs fictícios do laboratório simulado. Não é uso criptográfico; basta `usedforsecurity=False` .
- **Médio, B704 ( `Mazkup` ):** `output/html.py` marca como seguros o CSS/JS do próprio pacote. Os dados da rede entram como JSON com `</` escapado. Risco baixo, mas os nomes e descrições vindos dos equipamentos também chegam ao HTML da interface; vale um teste específico de XSS com hostname malicioso.
- **Médio, B104:** comparações com `0.0.0.0` em rotas, não abertura de socket. Falso positivo.
- **Baixo, B110/B406:** `except: pass` em pontos de limpeza e parser XML com `recover=True` para saídas de equipamento.

Os testes unitários passaram o tempo todo, mas nenhum deles pegou D0, D1, D2 ou D7. Todos dependem de dados reais: o diretório local mascarava o `.gitignore` , e os fixtures eram escritos à mão. A recomendação é versionar as saídas brutas deste laboratório ( `raw/` ) como fixtures de regressão.

## Segurança

- **API sem autenticação por padrão:** o token ( `NETT0P0_API_TOKEN` ) é opcional. Como a API recebe credenciais de rede, deveria ser obrigatório fora de `127.0.0.1` .
- **TLS e chaves de host:** `verify_ssl=False` por padrão ( `config.py:44` ), NETCONF com `hostkey_verify=False` ( `netconf.py:242` ) e SSH sem verificação de chave. Aceitável em laboratório, não em produção: permite interceptação das credenciais.
- **Segredos:** as credenciais ficam só em memória e não são gravadas (verificado no cenário 12). Falta integração com cofre (Vault, keyring) para uso agendado.
- **Dependência externa no relatório:** o HTML carrega o Cytoscape.js de CDN; em redes isoladas o diagrama não abre (as abas de documentação continuam funcionando).

## Limitações desta avaliação

- Três famílias de sistema: não há Cisco, Juniper, Huawei, Arista ou Fortinet reais neste laboratório.
- Escala pequena (8 equipamentos, 9 links); não mede comportamento com centenas de nós.
- SR Linux em contêiner e CHR em VM, não hardware físico; o NAT de gerência do CHR é específico do vnetlab e causa D6.
- Uma execução por cenário; como D1 mostrou, o resultado não é determinístico e merece repetição.
- Na primeira rodada, o `isp-pe` compartilhava o usuário SNMPv3 com os demais hosts e foi coletado indevidamente. Foi um erro de configuração do laboratório, corrigido antes da rodada final analisada aqui.

## Plano de melhorias

| PRIORIDADE | AÇÃO                                                                                                                                                           | RESOLVE     | ESFORÇO (ESTIMATIVA) |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------------------|
| 1          | Série só de fontes confiáveis; lista de valores genéricos ("Virtual Machine", "To Be Filled", "Default string"); exigir duas chaves fortes para fundir         | D1          | 1 dia                |
| 1          | Normalizar porta por <code>port-id</code> e não por descrição; fundir links pelo lado que coletou a própria interface; ignorar LLDP de <code>mgmt*</code> /OOB | D2, D5      | 2 dias               |
| 1          | Quando o IP anunciado falhar: resolver o hostname, procurar o MAC do chassis nas tabelas ARP coletadas e cruzar com o inventário                               | D3, D9      | 3 dias               |
| 1          | Identificar fabricante por banner/prompt SSH e testar drivers candidatos; mensagem de erro fiel                                                                | D4, D14     | 2 dias               |
| 2          | Parsers: registros RouterOS sem índice; SR Linux via <code>\  as json</code> ou JSON-RPC; <code>lldpcli</code> com caminho completo                            | D7, D8, D11 | 2 dias               |
| 2          | Ignorar IPs idênticos e interfaces de gerência na inferência L3; juntar vizinho BGP ao nó que tem aquele IP                                                    | D6, D12     | 1 dia                |
| 2          | Vizinhos OSPF/BGP por REST ( <code>/routing/ospf/neighbor</code> , <code>/routing/bgp/session</code> ) e série/modelo via OpenConfig <code>platform</code>     | D10, D13    | 2 dias               |

| PRIORIDADE | AÇÃO                                                                                                  | RESOLVE   | ESFORÇO (ESTIMATIVA) |
|------------|-------------------------------------------------------------------------------------------------------|-----------|----------------------|
| 2          | Fixtures de regressão com as saídas reais deste laboratório; laboratório rodando em cada pull request | qualidade | 2 dias               |
| 3          | Token obrigatório, verificação de TLS e de chaves de host, cofre de credenciais                       | segurança | 1 semana             |
| 3          | Laboratórios Huawei/Fortinet/VyOS (Redes Brasil) e Cisco (DevNet); teste de escala com 50-100 nós FRR | cobertura | 2 semanas            |
| 3          | Cytoscape embutido no HTML, layout para redes grandes, IPv6 e VRF                                     | produto   | 3-4 semanas          |

## Reprodução

```
# No GitHub: Actions → "Simulação em laboratório (Containerlab)" → Run workflow
# Resultados: artefato lab-results ou branch lab-results (runs/<id>/)
git clone -b lab-results https://github.com/tanatielly/network-discovery-automation resultados
python lab/scripts/score.py resultados/runs/36256925269-1          # métricas a partir dos JSONs
python docs/relatorios/build.py resultados/runs/36256925269-1 --pdf # regenera estes relatórios
```

Arquivos do run: `metrics.json` (métricas por cenário), `raw/` (saídas brutas e `parse-matrix.json`), `scenarios/<id>/` (topologia, eventos, logs e relatórios), `lab-state/` (estado dos equipamentos), `quality/` (testes, cobertura, mypy, bandit) e `logs/` (cada etapa do workflow).